

Reinforcement Learning: Fundamentals and MDP Framework

Stanford Lecture Notes

Abstract

These notes cover the fundamentals of reinforcement learning, including motivation, key challenges, and the Markov Decision Process (MDP) formalism. Topics include reward functions, the credit assignment problem, state transition probabilities, discount factors, and optimal policies.

Contents

1	Introduction to Reinforcement Learning	3
1.1	Motivation: When to Use RL	3
1.1.1	Example 1: Helicopter Flight Control	3
1.1.2	Example 2: Game Playing (Chess, Go)	3
2	Key Concepts in Reinforcement Learning	4
2.1	Reward Function	4
2.2	State	4
2.3	The Credit Assignment Problem	4
2.3.1	Example: Chess Game	4
2.3.2	Example: Self-Driving Car	4
3	Markov Decision Process (MDP) Formalism	5
3.1	MDP Definition	5
3.2	Concrete Example: Robot Navigation	5
3.2.1	Grid World Setup	5
3.2.2	State Transition Probabilities (Modeling Noise)	6
3.2.3	Reward Function Design	7
3.3	Execution Sequence	7
4	Total Payoff and Discount Factor	7
4.1	Total Payoff Formula	7
4.2	Understanding the Discount Factor γ	7
4.2.1	Effect of Discounting	8
4.2.2	Two Justifications for Using $\gamma < 1$	8
5	Policies and Optimal Solutions	8
5.1	Definition of a Policy	8
5.2	Executing a Policy	8
5.3	Optimal Policy	9

5.4	Example: Optimal Policy for Grid World	9
5.4.1	Non-Obvious Decision: North vs. West	9
5.5	Fine Distinctions in MDPs	9
6	The Reinforcement Learning Problem	10
6.1	Problem Statement	10
6.2	Design Steps	10
6.3	Learning State Transition Probabilities	10
7	Special Cases: Game Playing	10
7.1	Two-Player Games (Chess, Go)	10
8	Summary	11

1 Introduction to Reinforcement Learning

Reinforcement Learning (RL) is the fourth major topic in machine learning, alongside supervised learning, unsupervised learning, and other ML paradigms. RL is particularly useful when it is difficult or impossible to specify the "correct" action at every step.

1.1 Motivation: When to Use RL

Key Insight

Reinforcement learning is appropriate when you cannot easily specify a mapping from inputs X to outputs Y , but you can define when the system is performing well or poorly.

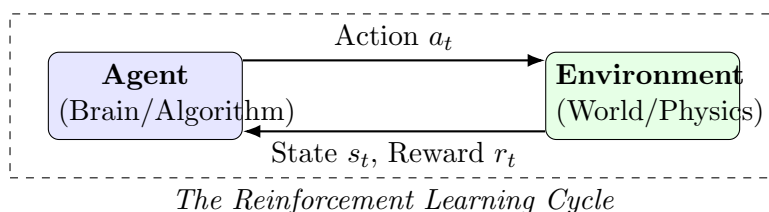


Figure 1: The Agent-Environment Interaction Loop. The agent takes actions, and the environment responds with the new state and a scalar reward.

1.1.1 Example 1: Helicopter Flight Control

Problem: Train a computer to fly a helicopter along a desired trajectory.

Challenge: At any given state (position, orientation, velocity), it is very difficult to specify the *one correct* control action. There is no clear supervised learning mapping $X \rightarrow Y$.

RL Solution: Instead of specifying exact actions, we define a **reward function** that:

- Gives high rewards when flying accurately on trajectory
- Gives large negative rewards when crashing or flying poorly

Analogy: Training a dog — you say "good dog" or "bad dog" rather than explaining each action in detail. The dog learns from positive and negative rewards.

1.1.2 Example 2: Game Playing (Chess, Go)

Problem: Create a program to play chess or Go.

Challenge: Given a board position, it is difficult to know the "one best move" definitively.

RL Solution: Let the program play games and provide rewards:

- $R = +1$ for a win
- $R = -1$ for a loss
- $R = 0$ for a tie

The algorithm learns how to maximize positive rewards through experience.

2 Key Concepts in Reinforcement Learning

2.1 Reward Function

The reward function $R(s)$ assigns numerical rewards to states:

$$R : S \rightarrow \mathbb{R}$$

where S is the set of possible states.

Designer's Role: Specify the reward function to encode the task objective.

2.2 State

The **state** represents the complete status of the system at any time:

- For chess: positions of all pieces on the board
- For helicopter: position, orientation, velocity, whether right-side up or inverted
- For robot navigation: current location in the environment

2.3 The Credit Assignment Problem

Critical Challenge

Credit Assignment Problem: When an algorithm receives a reward (positive or negative), how does it determine which prior actions were responsible for that outcome?

2.3.1 Example: Chess Game

Suppose a chess program loses on move 50:

- It receives reward $R = -1$
- But the critical mistake might have occurred at move 20
- It takes 30 additional moves before the outcome is sealed
- **Question:** Which moves were good? Which were bad?

2.3.2 Example: Self-Driving Car

If a self-driving car crashes:

- The action immediately before the crash might be "braking"
- But braking didn't *cause* the crash
- The problematic decision likely occurred many seconds earlier
- **Question:** What should the car do more of? What should it avoid?

Implication: RL algorithms must (at least indirectly) solve the credit assignment problem to learn effectively.

3 Markov Decision Process (MDP) Formalism

Reinforcement learning problems are formally modeled using **Markov Decision Processes (MDPs)**. This provides a mathematical framework for reasoning about sequential decision-making under uncertainty.

3.1 MDP Definition

An MDP is defined as a 5-tuple:

$$\text{MDP} = \langle S, A, P_{sa}, \gamma, R \rangle$$

where:

S	Set of States All possible configurations of the system • Chess: all possible board positions • Helicopter: all possible positions, orientations, and velocities
A	Set of Actions All possible actions the agent can take • Chess: all legal moves • Helicopter: all control stick positions • Robot: {North, South, East, West}
P_{sa}	State Transition Probabilities Probability distribution over next states given current state and action: $P_{sa}(s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$
γ	Discount Factor A number $\gamma \in [0, 1)$, typically $\gamma \approx 0.99$
R	Reward Function Maps states to rewards: $R : S \rightarrow \mathbb{R}$

3.2 Concrete Example: Robot Navigation

Consider a simple grid world where a robot navigates to reach a goal.

3.2.1 Grid World Setup

- **Grid:** 4×3 grid with one obstacle (wall)
- **States:** 11 possible positions (excluding the wall)
- **Actions:** $A = \{\text{North, South, East, West}\}$
- **Goal:** Navigate to cell $(4, 3)$ while avoiding cell $(4, 2)$

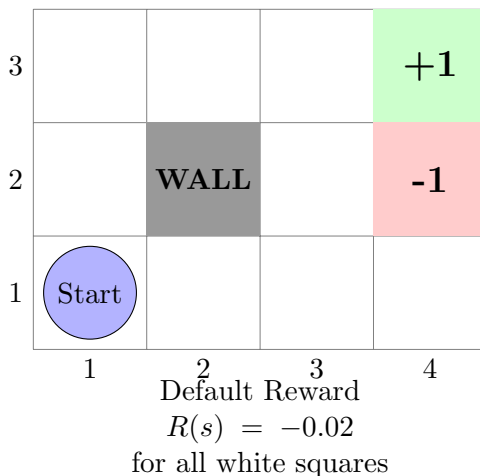


Figure 2: The 4x3 Grid World. The robot starts at (1,1) and must reach (4,3). Note the "danger state" at (4,2) and the obstacle at (2,2).

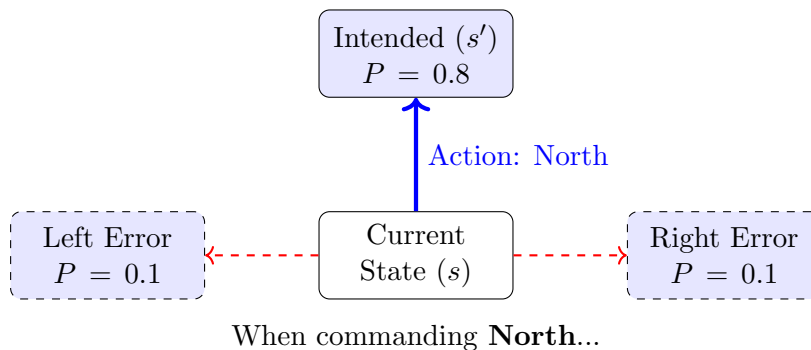


Figure 3: Stochastic Transition Model. The robot only goes in the intended direction 80% of the time.

3.2.2 State Transition Probabilities (Modeling Noise)

Real robots have noisy dynamics — wheels may slip, causing unintended movement.

Model: When commanding the robot to move in a direction:

- Probability 0.8: moves in intended direction
- Probability 0.1: veers to the left
- Probability 0.1: veers to the right

Example: From state (3, 1), commanding North:

$$P_{(3,1),\text{North}}(3,2) = 0.8$$

$$P_{(3,1),\text{North}}(4,1) = 0.1$$

$$P_{(3,1),\text{North}}(2,1) = 0.1$$

$$P_{(3,1),\text{North}}(s') = 0 \quad \text{for all other } s'$$

Walls: If the robot would hit a wall, it bounces back and stays in the current state.

3.2.3 Reward Function Design

To incentivize desired behavior:

$$\begin{aligned} R(4, 3) &= +1 && \text{(goal state)} \\ R(4, 2) &= -1 && \text{(danger state)} \\ R(s) &= -0.02 && \text{for all other states} \end{aligned}$$

Rationale for small negative reward:

- Penalizes "loitering" — using electricity unnecessarily
- Incentivizes reaching the goal quickly
- Models the cost of time and energy

3.3 Execution Sequence

The robot's trajectory through the MDP unfolds as follows:

1. Robot starts in initial state s_0 at time $t = 0$
2. Based on s_0 , chooses action a_0
3. Transitions to state $s_1 \sim P_{s_0 a_0}(\cdot)$
4. Based on s_1 , chooses action a_1
5. Transitions to state $s_2 \sim P_{s_1 a_1}(\cdot)$
6. Process continues: $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots$

4 Total Payoff and Discount Factor

4.1 Total Payoff Formula

The objective is to maximize the **expected total discounted payoff**:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \right]$$

where:

- $R(s_t)$ is the reward received at time t
- γ^t is the discount applied to future rewards

4.2 Understanding the Discount Factor γ

The discount factor $\gamma \in [0, 1)$ is typically set to values like 0.99.

4.2.1 Effect of Discounting

$$\text{Reward at } t = 0 : \gamma^0 \cdot R(s_0) = R(s_0)$$

$$\text{Reward at } t = 1 : \gamma^1 \cdot R(s_1) = 0.99 \cdot R(s_1)$$

$$\text{Reward at } t = 2 : \gamma^2 \cdot R(s_2) = 0.9801 \cdot R(s_2)$$

$$\text{Reward at } t = 3 : \gamma^3 \cdot R(s_3) = 0.970299 \cdot R(s_3)$$

Interpretation: Rewards further in the future are worth less than immediate rewards.

4.2.2 Two Justifications for Using $\gamma < 1$

1. Conceptual Justification:

- Encourages getting positive rewards sooner
- Encourages postponing negative rewards
- **Financial interpretation:** Time value of money
 - \$1 today > \$1 in one year (can earn interest)
 - Paying \$1 next year > paying \$1 today

2. Pragmatic Justification:

- Algorithms converge faster with $\gamma < 1$
- Easier to prove convergence properties
- Guarantees bounded total payoff (prevents infinite sums)
- If $\gamma = 1$, many RL algorithms may not converge

5 Policies and Optimal Solutions

5.1 Definition of a Policy

A **policy** π is a function that maps states to actions:

$$\pi : S \rightarrow A$$

Interpretation: The policy tells the agent which action to take in each state.

Alternative terminology: In robotics, policies are also called *controllers*.

5.2 Executing a Policy

To execute policy π :

- At each time step t
- Observe current state s_t
- Take action $a_t = \pi(s_t)$

5.3 Optimal Policy

The **optimal policy** π^* is the policy that maximizes the expected total discounted payoff:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \mid \pi \right]$$

5.4 Example: Optimal Policy for Grid World

For the robot navigation example described earlier, the optimal policy has been computed. Key observations:

- From state $(3, 1)$: $\pi^*(3, 1) = \text{West}$
- The policy navigates to $(4, 3)$ to collect the $+1$ reward
- The policy avoids $(4, 2)$ with the -1 penalty

5.4.1 Non-Obvious Decision: North vs. West

At certain states, the optimal action is not immediately obvious:

Option 1 — Go North:

- Shorter path to goal
- But 0.1 probability of slipping into the -1 state
- Risky

Option 2 — Go West:

- Longer path (more -0.02 penalties)
- Rewards more heavily discounted
- But safer — avoids risk of -1 penalty

Result: The optimal policy chooses the *longer, safer route*.

5.5 Fine Distinctions in MDPs

MDPs can capture subtle trade-offs:

- Risk vs. reward
- Speed vs. safety
- Short-term vs. long-term payoff

These trade-offs are encoded through the combination of:

- State transition probabilities P_{sa}
- Reward function $R(s)$
- Discount factor γ

6 The Reinforcement Learning Problem

6.1 Problem Statement

RL Problem Formulation

Given: An MDP $\langle S, A, P_{sa}, \gamma, R \rangle$

Find: The optimal policy π^* that maximizes expected total discounted payoff

Deploy: Execute π^* by taking action $\pi^*(s)$ whenever in state s

6.2 Design Steps

1. Formulate the problem as an MDP:
 - Define the state space S
 - Define the action space A
 - Specify state transition probabilities P_{sa}
 - Choose discount factor γ
 - Design reward function $R(s)$
2. Apply a reinforcement learning algorithm to compute π^*
3. Deploy the robot/agent using policy π^*

6.3 Learning State Transition Probabilities

Question: Where do the probabilities P_{sa} come from?

In the robot example, we assumed:

$$P_{s,\text{North}}(\text{intended}) = 0.8, \quad P_{s,\text{North}}(\text{left}) = 0.1, \quad P_{s,\text{North}}(\text{right}) = 0.1$$

In practice:

- We may not know exact values (e.g., 0.8 vs. 0.7)
- We may not know slip probabilities (e.g., 0.1 vs. 0.115)
- **Solution:** Use data to *learn* P_{sa} from experience

Algorithms for learning state transition probabilities will be covered in future lectures.

7 Special Cases: Game Playing

7.1 Two-Player Games (Chess, Go)

Two-player games introduce an additional complication:

State definition:

- State = board configuration *when it is your turn*
- After you move, opponent moves, then it's your turn again

- This completes one state transition

Uncertainty in state transitions:

- You don't know exactly what your opponent will do
- P_{sa} captures probability distribution over opponent's responses
- Better opponents lead to different P_{sa}

8 Summary

- **Reinforcement Learning** is used when we cannot easily specify correct actions but can define rewards
- **Key Challenge:** Credit assignment problem — determining which actions led to good or bad outcomes
- **MDP Formalism:** Problems are modeled as $\langle S, A, P_{sa}, \gamma, R \rangle$
- **Discount Factor:** Balances immediate vs. future rewards; aids convergence
- **Policy:** Function $\pi : S \rightarrow A$ that specifies actions
- **Goal:** Find optimal policy π^* that maximizes expected discounted reward
- **Next Steps:** Algorithms for computing optimal policies (e.g., Value Iteration, Policy Iteration, Q-Learning)